

The following code implements this rule:

---

```
for i in range(len(points_in_invalid)):
    for j in range(i + 1, len(points_in_invalid)):
        #count the number of times both of these are in the bad triangles
        count_occurrences = 0
        for k in range(len(invalid_triangles)):
            count_occurrences += 1 * (points_in_invalid[i] in
invalid_triangles[k]) * \                (points_in_invalid[j] in
invalid_triangles[k])
        if(count_occurrences == 1):
            delaunay.append(points_to_triangle(points_in_invalid[i],
points_in_invalid[j], \point_to_add))
```

---

Here we loop through every point in `points_in_invalid`. For each one, we loop through every following point in `points_in_invalid`. This double loop enables us to consider every combination of two points that was in an invalid triangle. For each combination, we loop through all the invalid triangles and count how many times those two points are together in an invalid triangle. If they are together in exactly one invalid triangle, then we conclude that they should be together in one of our new triangles, and we add a new triangle to our DT that consists of those two points together with our new point.

We have completed the steps that are required to add a new point to an existing DT. So we can take a DT that has  $n$  points, add a new point, and end up with a DT that has  $n + 1$  points. Now, we need to learn to use this capability to take a set of  $n$  points and build a DT from scratch, from zero points all the way to  $n$  points. After we get the DT started, it's really quite simple: we just need to loop through the process that goes from  $n$  points to  $n + 1$  points over and over until we have added all of our points.

There is just one more complication. For reasons that we'll discuss later, we want to add three more points to the collection of points whose DT we're generating. These points will lie far outside our chosen points, which we can ensure by finding the uppermost and leftmost points, adding a new point that is higher and farther left than either of those, and doing similarly for the lowermost and rightmost points and the lowermost and leftmost points. We'll se these points together as the first triangle of our DT. We'll start with a DT that connects three points: the three points in the new triangle just mentioned. Then, we'll follow the logic that we've already